FECAP – FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Curso de Ciência da Computação

6º Semestre — Turma 6º CCOMP

SMASH

ESPECIFICAÇÃO FORMAL DAS REGRAS DE FUNCIONAMENTO DA PAREDE INSTRUMENTADA DE TREINO DE TÊNIS

Entrega 2 — Teoria da Computação e Linguagens Formais

Projeto 7 — Grupo Smash

Felipe Vallim Soares — 24026060

Guilhermy Mariano Lisboa Garcia — 23025371

Gustavo Oliveira Demetrio — 24026213

Saulo Pereira de Jesus — 24026095

Kátia Milani Lara Bossi

São Paulo

2026

APRESENTAÇÃO DO PROJETO

O **Smash** transforma uma parede em uma superfície instrumentada de treino de tênis. Seis alvos de MDF, em uma grade de 3 colunas por 2 linhas, têm cada um uma pastilha piezoelétrica ligada a um pino analógico do ESP32. O firmware identifica o alvo atingido e a intensidade do golpe e envia o evento por WebSocket ao aplicativo. O aplicativo transforma esses eventos em jogo e registra cada sessão no backend. O LED da placa do ESP32 é o atuador, que responde ao jogador.

Na Entrega 1, modelamos o ciclo de detecção de impacto do firmware (M1). Desde então, a parede passou de 3×3 para 3×2, ganhou o atuador, e o aplicativo passou a conduzir a sessão de jogo, que é a segunda máquina deste documento (M2). Este documento descreve as regras de funcionamento do sistema como ele está implementado. As seções 1 a 6 seguem, na ordem, os seis itens do enunciado da Entrega 2.

Como se joga. Cada alvo tem uma de três cores (âmbar, azul ou vermelho), e cada cor aparece uma vez por linha da parede. Há quatro modos. No **Color**, o aplicativo pede uma cor por vez, e o jogador precisa acertar uma sequência de cores; um erro faz a sequência recomeçar. No **Score**, o aplicativo não pede cor: as cores da paleta do nível valem pontos, e acertar uma cor fora dela é erro e pode tirar pontos. **Infinite Color** e **Infinite Score** são as versões sem fim, que correm contra um relógio e terminam quando o tempo acaba. Color e Score têm 20 níveis cada, definidos em tabelas: no Color, cada nível define o tamanho da sequência, o prazo por alvo e o limite de erros; no Score, a pontuação-alvo, o tempo, a penalidade por erro e a precisão mínima.

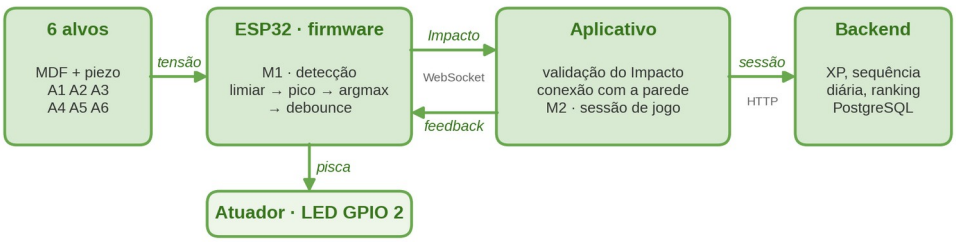


Figura 1 — Visão geral do sistema: onde cada máquina roda e o que trafega entre elas.

1 ATUALIZAÇÃO DA MÁQUINA DE ESTADOS

1.1 M1 — detecção de impacto (revisada)

O Quadro 1 resume o que mudou no projeto desde a Entrega 1.

Quadro 1 — Mudanças entre a Entrega 1 e a Entrega 2.

Aspecto	Entrega 1	Entrega 2
Alvos	9, em matriz 3×3	6, em grade 3×2
Leitura dos sensores	Multiplexador 74HC4051 em um pino	Cada piezo no seu pino: IO32, IO33, IO34, IO36, IO39 e IO35
Saída de M1	alvo 1..9, linha e coluna 1..3	alvo 1..6, linha 1..2, coluna 1..3
Atuador	Não havia	LED da placa (GPIO 2), acionado pela parede ou pelo app
Aplicativo	Apenas recebia o impacto	Conduz a sessão de jogo (M2)

Nenhuma dessas mudanças criou ou removeu uma fase do ciclo de detecção. Como na Entrega 1, M1 é um AFN-ε reconhecedor, equivalente a um AFD de três estados (Seção 6): $M1 = (Q_1, \Sigma_1, \delta_1, q_0, F_1)$, com $Q_1 = \{O, A, M, E, D\}$, $\Sigma_1 = \{s, v, q\}$, $q_0 = O$ e $F_1 = \{O\}$, e reconhece $L_1 = (svq)^*$, em que cada bloco svq é um ciclo completo de detecção. A Figura 2 mostra o diagrama revisado: as ações agora percorrem seis canais, e há uma saída nova: sem aplicativo conectado, cada impacto pisca o LED. A coluna Ação do Quadro 2 é a saída de cada transição.

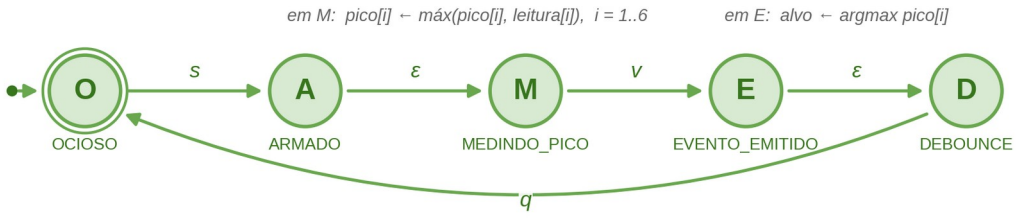


Figura 2 — Máquina de detecção de impacto (M1), revisada para 6 alvos.

Quadro 2 — Transições de M1.

Estado atual	Evento	Próximo estado	Ação
OCIOSO	s	ARMADO	Marca o início da janela e zera os seis picos
ARMADO	ε	MEDINDO_PICO	Começa a medir
MEDINDO_PICO	v	EVENTO_EMITIDO	Fecha a janela de 20 ms e elege o alvo de maior pico
EVENTO_EMITIDO	ε	DEBOUNCE	Envia o impacto ao app; sem app conectado, pisca o LED
DEBOUNCE	q	OCIOSO	Volta a aceitar batidas

1.2 M2 — sessão de jogo (nova)

O aplicativo agora conduz uma sessão de jogo, implementada como máquina de estados em `game-session.ts`. Os quatro modos de jogo usam a mesma máquina: cada modo só decide se a batida está certa e quando o nível termina. EM JOGO agrupa os estados em que o nível corre; os de borda dupla são finais.

Formalmente, M2 é um transdutor de Mealy com estados finais, M2 = (Q_2, Σ_2, Γ_2, δ_2, λ_2, q_0, F_2). Q_2 são os nove estados do Quadro 3, Σ_2 = {c, h, e, x, z, f, r, p, t, d, k} e q_0 = INICIANDO. Γ_2 = {acerto, erro, nível concluído, nível falhou} são os sinais de feedback enviados ao LED e à vibração do celular, e λ_2: $Q_2 \times \Sigma_2 \rightarrow \Gamma_2^*$ associa h a acerto; e e x a erro; z a acerto seguido de nível concluído; f a erro seguido de nível falhou, quando a falha vem de uma batida ou de um prazo, ou só a nível falhou, quando vem do tempo do nível. Os demais eventos não produzem saída (ε). F_2 = {NIVEL_CONCLUIDO, NIVEL_FALHOU}.

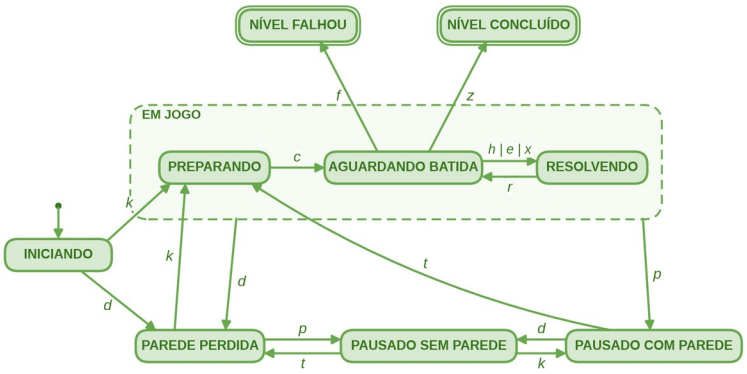


Figura 3 — Máquina da sessão de jogo (M2).

Quadro 3 — Estados de M2.

Estado	O que representa
INICIANDO	O nível acabou de abrir; lê se a parede está conectada (`startSession`)
PREPARANDO	Contagem de 3 s antes do primeiro alvo, ou depois de uma pausa ou queda
AGUARDANDO_BATIDA	Relógio correndo; é o único estado em que a batida conta
RESOLVENDO	Mostra o resultado da última batida antes do próximo alvo
PAUSADO_COM_PAREDE	O aplicativo foi para o segundo plano com a parede conectada
PAUSADO_SEM_PAREDE	O aplicativo está em segundo plano e a parede não está conectada
PAREDE_PERDIDA	A conexão com a parede caiu, ou não existia quando o nível começou
NIVEL_CONCLUIDO	Final: o jogador cumpriu o objetivo do nível
NIVEL_FALHOU	Final: acabaram as tentativas ou o tempo do nível

Quadro 4 — Transições de M2.

Estado atual	Evento	Próximo estado
INICIANDO	k — parede conectada · d — sem parede	PREPARANDO · PAREDE_PERDIDA
PREPARANDO	c — contagem de 3 s terminou	AGUARDANDO_BATIDA
AGUARDANDO_BATIDA	h — acerto · e — erro · x — prazo esgotado	RESOLVENDO
AGUARDANDO_BATIDA	z — acerto que completa o nível	NIVEL_CONCLUIDO
AGUARDANDO_BATIDA	f — falha (erros ou tempo esgotados)	NIVEL_FALHOU
RESOLVENDO	r — fim do tempo de feedback	AGUARDANDO_BATIDA
EM JOGO	d — conexão com a parede caiu	PAREDE_PERDIDA
PAREDE_PERDIDA	k — conexão voltou	PREPARANDO
EM JOGO · PAREDE_PERDIDA	p — app vai para segundo plano	PAUSADO_COM_PAREDE · PAUSADO_SEM_PAREDE
PAUSADO_COM_PAREDE	d — conexão com a parede caiu	PAUSADO_SEM_PAREDE
PAUSADO_SEM_PAREDE	k — conexão voltou	PAUSADO_COM_PAREDE
PAUSADO_COM_PAREDE · PAUSADO_SEM_PAREDE	t — app volta	PREPARANDO · PAREDE_PERDIDA

Os eventos h, e, x, z e f são **mutuamente exclusivos**: a regra do modo classifica cada batida, ou cada prazo esgotado, em exatamente um deles. Uma batida certa que completa o nível é z, e não h. Um erro ou um prazo esgotado que passa do limite de erros é f, e não e ou x. A falha por tempo do nível só é verificada em AGUARDANDO_BATIDA: se o tempo acaba durante RESOLVENDO, f ocorre logo depois do feedback. Nos modos Infinite, toda partida termina em NIVEL_FALHOU por tempo: esse é o fim normal da partida.

No código, PAUSADO_COM_PAREDE e PAUSADO_SEM_PAREDE são um único valor, `paused`, mais o campo `deviceStatus` da sessão, que `resume` consulta ao voltar. Um evento sem linha no Quadro 4 é ignorado, e o estado se mantém, como uma batida em PREPARANDO (R10). No código, Σ_2 chega por quatro entradas de `applyInput`: `deviceEvent` (batidas, d e k), `tick` (c, r, x e a falha por tempo), `pause` (p) e `resume` (t). d e k vêm do ciclo de conexão do aplicativo com a parede, que não foi detalhado: para o jogo, só importa se ela está conectada.

Uma falha encontrada pela modelagem. A primeira versão do Quadro 4 mostrou que, se a parede caísse e voltasse com o app em segundo plano, a contagem começava com o celular bloqueado. O código foi corrigido para o comportamento acima, com testes automatizados.

2 DESCRIÇÃO DOS EVENTOS

Os eventos do sistema vêm de quatro origens: o sensor, o tempo, a rede e o jogador. O Quadro 5 relaciona todos eles e o que cada um provoca.

Quadro 5 — Eventos do sistema.

Símbolo	Evento	Origem	O que provoca
s	A leitura de um alvo cruza o limiar daquele alvo	Sensor	M1: OCIOSO → ARMADO
v	A janela de medição de 20 ms termina	Tempo	M1: elege o alvo de maior pico
q	O debounce de 200 ms termina	Tempo	M1: DEBOUNCE → OCIOSO
—	O app se conecta à parede	Rede	A parede informa a versão do protocolo e o número de alvos; o app envia o id da sessão
—	Batimento de status	Tempo	A parede avisa a cada 2 s que está viva; 7 s sem mensagem é conexão perdida
—	Comando de feedback	App	A parede pisca o LED: acerto, erro, nível concluído ou nível falhou
—	Mensagem inválida	Rede	Registrada como falha; nenhum estado muda
c	A contagem de 3 s termina	Tempo	M2: começa a aguardar a batida
h	Batida no alvo certo que não completa o nível	Parede	M2: registra acerto
e	Batida no alvo errado, dentro do limite de erros	Parede	M2: registra erro
x	O prazo para bater esgota, dentro do limite de erros (só no Color)	Tempo	M2: conta como erro
z	Acerto que completa o nível	Parede	M2: nível concluído
f	Erro ou prazo que passa do limite de erros, ou tempo do nível esgotado	Parede ou tempo	M2: nível falhou
r	O tempo de feedback da batida termina	Tempo	M2: volta a aguardar a batida
p / t	O app vai para segundo plano / volta	Jogador	M2: pausa / retoma, com contagem ou à espera da parede
d / k	A conexão com a parede cai / volta (ou o seu estado quando o nível abre)	Rede	M2: interrompe o nível / nova contagem; em pausa, troca entre os dois estados de pausa
—	O nível termina	App	A sessão é enviada ao backend, se o jogador estiver logado

Os eventos marcados com — não fazem parte do alfabeto das máquinas. Eles tratam da comunicação entre a parede, o aplicativo e o backend, e só chegam a M2 como uma batida válida ou como os eventos d e k.

3 REGRAS DE FUNCIONAMENTO

As regras abaixo controlam o comportamento do sistema. Cada uma corresponde a uma verificação no código.

- **R1.** Uma batida só é detectada se a leitura de algum alvo cruzar o limiar daquele alvo; cada alvo tem o seu limiar.
- **R2.** Se mais de um alvo vibrar na mesma batida, vale o de maior pico na janela de 20 ms, e não o primeiro a cruzar o limiar. Em caso de empate, vence o alvo de menor número.
- **R3.** Cada batida gera exatamente um evento. Nos 200 ms seguintes, nenhuma nova batida é aceita.
- **R4.** Se nenhum app estiver conectado, a parede pisca o LED a cada batida. Se houver app conectado, o LED só acende por comando do app.
- **R5.** Se a parede receber um comando inválido, ela responde com uma mensagem de erro e nada muda.
- **R6.** Se uma mensagem não seguir o contrato `Impacto` (campo faltando, alvo fora de 1 a 6, intensidade fora de 0 a 4095), ela não vira batida.
- **R7.** Uma batida repetida, ou mais antiga que a última aceita, é descartada.
- **R8.** Se a parede usar outra versão de protocolo ou outro número de alvos, o app sinaliza parede incompatível.
- **R9.** Se a parede ficar 7 s sem enviar nada, a conexão é dada como perdida e o app tenta reconectar a cada 0,5 a 8 s.
- **R10.** Uma batida só conta em AGUARDANDO_BATIDA; na contagem, no feedback, pausado ou sem parede, ela é ignorada.
- **R11.** Se a conexão com a parede cair, o nível é interrompido, mas não reprovado. Quando ela volta, há uma nova contagem.
- **R12.** Com o app em segundo plano, o nível fica pausado mesmo que a parede caia ou volte. Ao retomar, há nova contagem se a parede estiver conectada; se não, o nível espera a parede.
- **R13.** No Color, um erro faz a sequência de cores recomeçar do início.
- **R14.** Um nível concluído ou falhado não muda mais de estado.
- **R15.** O XP é calculado pelo servidor; o app só informa o que aconteceu na sessão. Uma sessão com dados por alvo incoerentes com os totais é rejeitada.

4 PRÉ E PÓS-CONDIÇÕES

O Quadro 6 define as pré e pós-condições das principais operações do sistema.

Quadro 6 — Pré e pós-condições das principais operações.

Operação	Pré-condições	Pós-condições
Detectar impacto	Parede em OCIOSO. Limiares calibrados. A leitura de algum alvo cruzou o limiar.	Um único impacto com alvo de 1 a 6, linha de 1 a 2, coluna de 1 a 3 e intensidade de 0 a 4095. Parede em DEBOUNCE.
Conectar o app à parede	App e parede na mesma rede Wi-Fi. Parede ligada, em `ws://192.168.4.1:81`.	Conexão aberta. A parede informou protocolo e número de alvos. O app enviou o id da sessão.
Validar mensagem recebida	Conexão aberta. Mensagem de texto recebida.	A mensagem virou uma batida válida ou uma falha registrada. O app nunca trava por causa dela.
Registrar batida no jogo	M2 em AGUARDANDO_BATIDA. Batida válida.	Tentativa registrada e pontuação atualizada. M2 em RESOLVENDO, NIVEL_CONCLUIDO ou NIVEL_FALHOU.
Acionar o LED	Conexão aberta. Resultado: acerto, erro, nível concluído ou nível falhou.	O LED pisca o padrão do resultado. Nenhum comando fica em fila.
Registrar a sessão no backend	Jogador logado. M2 em estado final. Dados por alvo coerentes com os totais.	Sessão gravada no banco. XP calculado pelo servidor e somado ao jogador.

5 MODELO COMPUTACIONAL

O modelo que melhor representa o Smash é o **Autômato Finito**. M1 é usado como reconhecedor da linguagem L_1 = (svq)*, e as duas máquinas funcionam como transdutores finitos (máquinas de Mealy), com saídas nas transições: enviar o impacto, piscar o LED, registrar a tentativa. O conjunto de estados é finito e fixo, declarado no código como um tipo: `WallState` no firmware e `SessionPhase` no aplicativo. O próximo estado depende do estado atual e do evento recebido, e nenhuma transição consulta o histórico de eventos anteriores.

A sessão de jogo também guarda contadores: a posição na sequência, os erros, a pontuação e as tentativas. Alguns não têm limite: os primeiros níveis do Color aceitam erros sem limite, a pontuação dos modos infinitos cresce sem fim, e a precisão mínima do Score compara acertos com tentativas. Por isso M2 não modela a contagem. M2 modela o **controle da sessão sobre um alfabeto já classificado**: a regra de cada modo é uma função que recebe a batida e os contadores, sempre termina e devolve um único símbolo (h, e, z ou f). M2 só reage a esse símbolo. A contagem fica na regra do modo, e as fases da sessão, que são finitas, ficam no autômato.

Por que não um Autômato com Pilha. A pilha é necessária quando a linguagem tem aninhamento sem limite, como parênteses balanceados. Nada no comportamento do Smash tem essa forma: uma batida não abre algo que precise ser fechado depois. As mensagens trocadas com a parede são objetos JSON planos, sem objetos dentro de objetos.

Por que não uma Máquina de Turing. O controle do sistema não precisa de memória ilimitada: as fases são finitas, e a detecção guarda só seis picos e os identificadores dos 32 últimos impactos (para descartar repetidos). Com estados finitos, é possível listar todos os pares (estado, evento) e conferir que cada um tem um tratamento definido, como fazem os Quadros 2 e 4. Um modelo mais poderoso tornaria essa verificação mais difícil, sem descrever nada que o autômato finito já não descreva.

6 ANÁLISE DE DECIDIBILIDADE

O Quadro 7 relaciona as principais decisões de resposta SIM ou NÃO que o sistema toma e mostra por que cada uma sempre termina.

Quadro 7 — Decisões tomadas pelo sistema.

Decisão	Onde	Por que sempre termina
A leitura de algum alvo cruzou o limiar?	Firmware	No máximo 6 comparações
O alvo i tem o maior pico da janela?	Firmware	Compara o pico de i com os outros 5; no empate, vence o de menor número
O comando recebido é válido?	Firmware	Compara com os 3 comandos (session, ping, feedback) e, no feedback, com os 4 resultados (acerto, erro, nivel_concluido, nivel_falhou)
A mensagem segue o contrato `Impacto`?	App	Confere 6 campos, cada um contra um tipo e uma faixa
A batida é repetida ou antiga?	App	Busca entre os 32 últimos impactos aceitos e compara o tempo
A batida está certa?	App	Color: compara a cor do alvo com a cor pedida. Score: verifica se a cor está na paleta do nível
O nível foi concluído ou falhou?	App	Compara contadores e tempo com os limites do nível
A sessão enviada é coerente?	Backend	Soma os dados de no máximo 6 alvos e compara com os totais

Exemplos de entradas reais e da resposta que o sistema dá:

- `{"cmd":"feedback","resultado":"acerto"}` chega à parede → **SIM**, o comando é válido: o LED acende por 250 ms.
- `{"cmd":"feedback","resultado":"talvez"}` chega à parede → **NÃO**: ela responde "resultado desconhecido" e nada muda (R5).
- Uma mensagem com `"alvo":7` chega ao app → **NÃO**, não segue o contrato `Impacto`: vira uma falha registrada e não conta como batida (R6).

Todas essas decisões são decidíveis: a entrada é finita, o número de passos tem um limite fixo, e nenhuma delas fica esperando uma condição externa para responder.

Elas também são **determinísticas**: a mesma entrada produz sempre a mesma resposta. Em M1, cada transição ε é a única saída do seu estado e não tem condição. Juntando os estados ligados por ε, M1 vira um autômato determinístico de três estados, {O}, {A, M} e {E, D}, com a mesma linguagem (svq)*. Em M2, os eventos h, e, x, z e f são mutuamente exclusivos, e a transição é uma função pura, `applyInput(sessão, entrada, regras, nível)`, que não lê relógio nem rede: o tempo chega como entrada.

CONCLUSÃO

O comportamento do Smash na versão 3×2 é descrito por duas máquinas de estados finitas. M1 detecta o impacto na parede e continua igual à Entrega 1: só os parâmetros e as saídas mudaram com a nova montagem. M2 conduz a sessão de jogo no aplicativo e é a mesma para os quatro modos. As regras, as pré e pós-condições e as decisões descritas aqui correspondem ao código implementado, e todas as decisões do sistema são decidíveis e determinísticas. A própria modelagem apontou uma falha no comportamento da pausa, que foi corrigida no código antes desta entrega.